5 Rule-Based Modeling

Single Plants Are "Emerging"

Aristid Lindenmayer's approach to describing morphological forms of plants using so-called string rewriting systems [117, 118, 119] opened a broad scientific field in botany as well as in computer graphics. Text or string rewriting systems are subsets of rule-based systems, which have been analyzed for quite a while as solutions to problems in computer science. In a rule-based system, contrary to a procedural method, a formal rule basis is used to transform an initial state into a final state by applying a number of changes. Often, this provides an extremely compact description for complex final conditions. However, the process of generating geometrical data from a rule set is an elaborate one. Only recently, due to improved computer efficiency, the vast amount of data needed for larger objects can be managed more or less interactively.

In a string rewriting system, rules are used to operate on a string consisting of letters of a certain alphabet. The rules hereby generally increase the length of the string. To produce plant geometry, the characters of the string are graphically interpreted using a so-called turtle metaphor. Many aspects of rule-based modeling, in particular the use of the Lindenmayer systems for the production of plant geometry, are examined more thoroughly in Prusinkiewicz and Lindenmayer's book "The Algorithmic Beauty of Plants"[166], a classic in computer graphics.

Along with string rewriting, there are also other representational forms for rule-based systems. Thus, after introducing the Lindenmayer systems we will discuss iterated function systems, whose "rule-systems" are sets of affine transformations that are applied to a given point set. The result, in this case, is a set of points that, provided there is an appropriate parameterization, is an image of a natural object, and thus not its geometry.

Graphs with appropriate processing methods can also describe rule systems. The so-called object instancing paradigm, for example, permits the creation of a subset of the Lindenmayer systems as well as the synthesis of a subset of iterated function systems over graphs. The object instancing is an important preliminary stage for the rule-based object production that will be described in the next chapter. Finally, we will introduce another system that constructs a so-

called CSG graph[1] using Lindenmayer systems; the graph is then used to create geometry. This method extends the idea of the graph-based representation of geometry.

5.1 Rewriting Systems

von Koch curve →

The general mechanism of a rewriting system can best be illustrated with the so-called snowflake curve or von Koch curve. This curve is actually the classic example of a rewriting system, and, therefore, it is found in many places in the computer science literature. Rewriting takes place graphically: each edge of a given geometry is replaced by a sequence of edges. The rewriting method is implemented using a so-called generator (see Fig. 5.1a). The successive application of generators on the edges of the initial object, the initiator, results in a complex figure, which resembles here the outline of a snowflake.

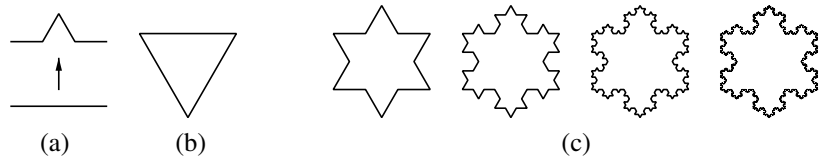


Figure 5.1
Graphical production of the von Koch curve: (a) generator; (b) initiator; (c) illustrations after 1, 2, 3 and 7 rewritings

As already mentioned, graphical rewriting is just one option for the definition of a rewriting system. There are a number of different methods used for various purposes:

- **Graphs**

  Here edges and nodes of a graph are replaced by subgraphs (see [27]). The original graph is in this way enlarged step by step. The underlying mechanism uses grammars that specify the rewriting rules. In Sect. 5.13 a similar method is outlined that works, however, with textual rules.

- **Cell Grids**

  Values or value combinations in a cell are replaced with other values. An example is "The Game of Life" (see [70, 71]), which produces complex spatial patterns using simple rules (see also Sect. 4.1). Similar mechanisms are used for the modeling of plant populations and other processes [116].

- **Text**

  In the already mentioned L-systems, characters in a text are replaced. The mechanism was introduced by Thue (see [185]). Chomsky [26], a linguist, compiled a concept for the description of languages using so-called formal grammars with alphabets, axioms and productions, which incidentally is also the basis for the Lindenmayer systems.

[1] CSG stands for constructive solid geometry.

5.2 Lindenmayer Systems

Lindenmayer systems or L-systems, are one kind of string rewriting mechanism, consisting of a set of rules and symbols that model growth processes. An initial string or symbol, the axiom, is defined; one or more rules are added that replace the character string in a given alphabet. In an L-system, rules are executed in parallel: per step all applicable rules are also applied to the text.

Lindenmayer defined the system this way to model the parallel development of naturally occurring phenomena in biological structures, such as in cells or shoots. A classic example in the area of cell division further elucidates the method: the development of filaments of the bacteria *Anabaena Catenula* and of various algae can be described using two cytological conditions a and b that represent size and readiness for division (see also [166]). After the first cell division, the subcells must grow first before they divide again. Additionally, each cell owns a polarity that describes the new position resulting from the division of the new (daughter) cells. The polarity is given by r and l and alternates in a fixed pattern during cell division. The growth and polarity of the cells can be derived using the following rule set:

$$\{a_r ::= a_l b_r, a_l ::= b_l a_r, b_r ::= a_r, b_l ::= a_l\}.$$

From the axiom a_r results, by application of the rules,

$$a_r \rightarrow a_l b_r \rightarrow b_l a_r a_r \rightarrow a_l a_l b_r a_l b_r \rightarrow b_l a_r b_l a_r a_r b_l a_r a_r \rightarrow \ldots,$$

and hence the typical pattern of the filaments. The parallelism of the replacement is the essential difference between the L-systems and, for example, the procedural methods. With the L-system, a variety of rule mechanisms such as the exchange of signals can be modeled, which is otherwise difficult to achieve.

← parallel replacement

An L-system is defined using a so-called formal grammar $G = (V, \omega, P)$, where V represents the alphabet, ω a nonterminal string (axiom), and P is a finite set of productions (replacement rules). The set V^* represents the set of strings, the set V^+ is the set of all nonempty strings. The production $p \in P$ is written by $\phi ::= \chi$, whereby $\phi, \chi \in V^+$. In a context-free L-system (0L-system), ϕ has the length one; the rule is thus also applicable wherever the character $a = \phi$ appears.

Contrary to context-free L-systems in which the production rules take account only of an individual symbol, in a context sensitive L-system, a rule depends also on its neighbors, e.g., on the characters in the surrounding text. For example, $\phi = \tau a \upsilon$, where $\tau, \upsilon \in V^+$ and $a \in V$. If τ has the length k and υ the length l, we speak of a (k,l)-system. Often 2L-systems are used with $k = l = 1$ and 1L-systems with $k = 1, l = 0$ or $k = 0, l = 1$; here the context is taken into account only on one side of a.

In a deterministic L-system there exists for one ϕ at the most one rule with ϕ on the left side, e.g., for each symbol in the alphabet there is exactly one production. Thus in each case one rule can be applied. For many applications

deterministic context-free L-systems (D0L systems) are sufficient. If random processes are needed, one uses stochastic L-systems, described by the grammar $G = (V, \omega, P, \pi)$, which, in addition to the deterministic L-system, assigns to the production set P a set of probabilities $\pi : p \rightarrow (0, 1]$. If a ϕ is found on the left side of one or several rules, the rule is selected from the set of the possible rules and used according to its probability.

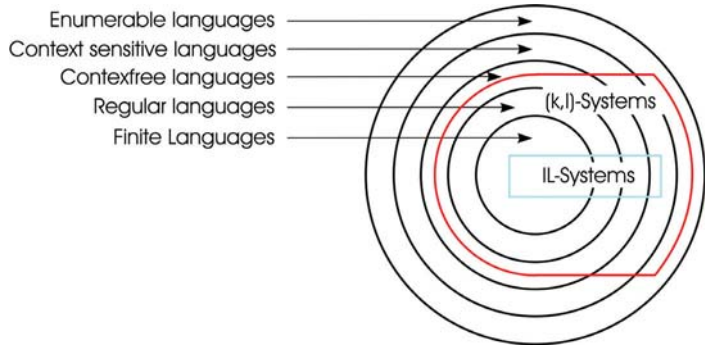


Figure 5.2
Power of 0L-systems and (k,l)-systems in comparison to Chomsky languages

The opposite to the parallel rule mechanisms and the otherwise customary sequential methods of Chomsky grammars is expressed in the different modeling powers of L-systems in contrast to the Chomsky languages. Modeling power represents in this context the set of denotable strings. Figure 5.2 shows the power of context-free 0L-systems and context-sensitive L-systems ((k,l)-systems) in comparison to Chomsky classes of languages.

For another example of L-systems, the alphabet $V = \{f, F, +, -\}$, the axiom $w = F - -F - -F$, and a rule set with only one rule $P = \{F ::= F + F - -F + F\}$ is provided. In this case, the rule is applied as often as F exists in the string. The derivation results in the following strings:

1. Step: $F - -F - -F$

2. Step: $F+F--F+F--F+F--F+F--F+F--F+F$

3. Step: $F + F - -F + F + F + F - -F + F - -F + F-$
$-F + F + F + F - -F + F - -F + F - -F+$
$F + F + F - -F + F - -F + F - -F + F + F$
$+F - -F + F - -F + F - -F + F + F + F - -F + F$

The developed pattern can now be interpreted in different ways. In the above example of the filaments, Lindenmayer uses the states of the cells in the fiber to directly produce graphic data. Alternatively a second step can be switched behind the string replacement that interprets the text, so that the geometry is generated.

Among the different methods that execute this form of graphical interpretation, the turtle metaphor [1] has gained general acceptance. It was invented by Prusinkiewicz [162]. Here a virtual turtle is moved over the drawing plane or

through a given three-dimensional space. The state of the turtle is the vector (x, y, α) that represents the position and angle of the direction in which the turtle is moving. During the movement the turtle must in each case again be aligned actively, otherwise it keeps moving in the initially chosen direction. Genuine turtles are said to move similarly: if an initial position and direction are given, they usually move in a straight line, until a change of direction becomes necessary. The movement of the computer turtle is affected completely analogously using the following four commands:

F Move turtle about d into current direction, draw a line:
$(x, y, \alpha) \rightarrow (x + d\cos\alpha, y + d\sin\alpha, \alpha)$

f Move turtle about d into current direction, without drawing a line:
$(x, y, \alpha) \rightarrow (x + d\cos\alpha, y + d\sin\alpha, \alpha)$

+ Increase current angle by angle δ:
$(x, y, \alpha) \rightarrow (x, y, \alpha + \delta)$

– Decrease current angle by angle δ:
$(x, y, \alpha) \rightarrow (x, y, \alpha - \delta)$

For a complete specification, the length d and the angle δ still have to be defined. For $d = 1$ and $\delta = 60°$ the axiom of the L-system already described above transforms into an equilateral triangle. The application of the production rule transforms each line into a path that conforms with the generator in the von Koch curve. Accordingly, the strings in the second step produce the first "snowflake" in Fig. 5.1c and the following steps each refine the image of the curve.

5.3 Branching Structures

For the modeling of branching structures, L-systems have to be extended. The processing of the string sequence is now accomplished by means of a so-called pushdown automaton. This method contains the possibility to store, and at another time again to recall the state of the turtle in a stack. This occurs in such a way that the last stored state is always the first to be read.

← stack (data structure)

In order to implement a pushdown automaton during evaluation of a string produced by an L-system, the current state of the system (x, y, α) must be stored by means of the appropriate stack data structure, and it must also be loadable again. In the text, these operations are represented by two new characters:

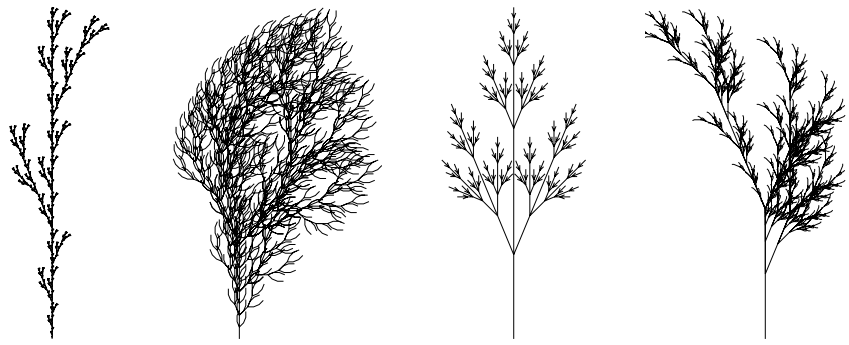
← pushdown automaton

[store current state (x, y, α) on the stack

] load state (x, y, α) from stack

The use of a pushdown automaton for branching systems has the advantage that the state of the turtle can be stored before a branching, and after the associated subtree has been drawn it can easily be reloaded again.

Additionally, for modeling branching structures the L-systems operate on an extension of the so-far used alphabet $V = \{F, f, +, -, \alpha, [,]\}$ by adding more characters that are used for the expansion but, however, are ignored in the graphical interpretation; n is again the number of replacement steps.



(a) (b) (c) (d)

Figure 5.3
Branching structures using Lindenmayer systems: (a)–(b) edge rewriting; (c)–(d) node rewriting

Part	n	δ	ω	P
(a)	5	25.7°	F	$F ::= F[+F]F[-F]F$
(b)	4	22.5°	F	$F ::= FF - [-F + F + F] + [+F - F - F]$
(c)	7	25.7°	X	$X ::= F[+X][-X]FX, F ::= FF$
(d)	5	22.5°	X	$X ::= F[[X] - X]F[+FX] - X, F ::= FF$

The systems shown in Fig. 5.3 are generated by two different production methods. The systems in parts (a) and (b) result from expansion of the edges (commands F). This is recognized by the fact that in the corresponding productions an F is replaced by a branching. The systems in parts (c) and (d) use the extra letter X that does not have a graphic meaning; it can be interpreted as a node from which a branching evolves.

When analyzing the systems, a problem connected with L-systems becomes evident: their relatively abstract specification. The differences in the forms of the modeled shrubs can not be recognized in the rule systems. Therefore, not only is it difficult to model a specific, given form with an L-system, but it also requires a lot of experience. A further complication is the local modeling methodology, which L-systems indeed also share with some procedural methods. The total form of a plant is determined here using local angles and distances. Small alterations of these parameters can dramatically modify the total shape of a plant.

5.4 Three-Dimensional Commands

The commands operated so far on a two-dimensional plane. However, they can easily be extended to a three-dimensional space. Aside from the third coordi-

nate, the only difference is the more complex definition of angles and rotation axes for the movement of the turtle.

During the course of the movement, the coordinate system is so to speak positioned at the head of the turtle, i.e., the turning just like the shifting affects the turtle locally. Here the coordinate system can be turned relative to three axes. If the coordinate system is positioned to the head of the turtle so that in fact it always moves in the z direction, then the rotations represented in Fig. 5.4 can be executed.

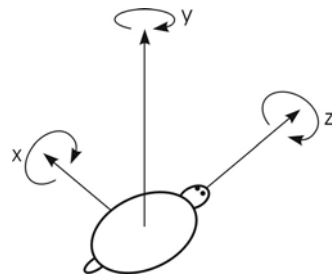


Figure 5.4
Local coordinate system for controlling the turtle in three dimensions

Accordingly the divergence angle δ is used either corresponding to the x-, y- or to the z-axis. The rotations about theses axes are defined by rotation matrices R_x, R_y, and R_z:

$$R_x(\delta) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos\delta & -\sin\delta \\ 0 & \sin\delta & \cos\delta \end{pmatrix} \qquad R_y(\delta) = \begin{pmatrix} \cos\delta & 0 & -\sin\delta \\ 0 & 1 & 0 \\ \sin\delta & 0 & \cos\delta \end{pmatrix} \qquad R_z(\delta) = \begin{pmatrix} \cos\delta & \sin\delta & 0 \\ -\sin\delta & \cos\delta & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

The machine is now described by the state vector $(x, y, z, \mathbf{M})$, where $\mathbf{M}$ is a rotation matrix. In aviation jargon the z-axis is called the course axis, the y-axis, the vertical axis, and the x-axis, the lateral axis. The rotations are called rolling, heading, and pitching. The appropriate commands are now:

F Move turtle at $\vec{d}$ in current direction, draw line:
$(x, y, z, \mathbf{M}) \rightarrow (x + (\mathbf{M}\vec{d})_1, y + (\mathbf{M}\vec{d})_2, z + (\mathbf{M}\vec{d})_3, \mathbf{M})$

f Move turtle at $\vec{d}$ in current direction, without drawing line:
$(x, y, z, \mathbf{M}) \rightarrow (x + (\mathbf{M}\vec{d})_1, y + (\mathbf{M}\vec{d})_2, z + (\mathbf{M}\vec{d})_3, \mathbf{M})$

\+ Increase the angle in terms of vertical axis at δ:
$(x, y, z, \mathbf{M}) \rightarrow (x, y, z, \mathbf{M} \cdot \mathbf{R_y}(\delta))$

– Decrease the angle in terms of vertical axis at δ:
$(x, y, z, \mathbf{M}) \rightarrow (x, y, z, \mathbf{M} \cdot \mathbf{R_y}(-\delta))$

& Increase the angle in terms of lateral axis at δ:
$(x, y, z, \mathbf{M}) \rightarrow (x, y, z, \mathbf{M} \cdot \mathbf{R_x}(\delta))$

$\wedge$ Decrease the angle in terms of lateral axis at δ:
$(x, y, z, \mathbf{M}) \rightarrow (x, y, z, \mathbf{M} \cdot \mathbf{R_x}(-\delta))$

\ Inrease the angle in terms of roll axis at δ:
$(x, y, z, \mathbf{M}) \rightarrow (x, y, z, \mathbf{M} \cdot \mathbf{R_z}(\delta))$

/ Decrease the angle in terms of roll axis at δ:
$(x, y, z, \mathbf{M}) \rightarrow (x, y, z, \mathbf{M} \cdot \mathbf{R_z}(-\delta))$

| Turn around, here use matrix $R_y(180°)$:
$(x, y, z, \mathbf{M}) \rightarrow (x, y, z, \mathbf{M} \cdot \mathbf{R_y}(180°))$

A small shrub shall serve as an example for the modeling with three-dimensional commands. The L-system consists of the above-mentioned alphabet and four productions ($n = 7, \delta = 22.5°, w = A$):

$$
\begin{aligned}
A &::= [\&FL!A]/////'[\&FL!A]///////'[\&FL!A] \\
F &::= S/////F \\
S &::= FL \\
L &::= ['''\wedge\wedge\{-f+f+f-|-f+f+f\}]
\end{aligned}
$$

The curved brackets serve here for marking a path in the form of a point sequence. These points are triangulated during the geometry production to triangles; the triangles then defines the surface. The symbols ! and $'$ serve for the reduction of the branch diameter and for the control of the colors. The result is displayed in Fig. 5.5. It shows a small three-dimensional shrub with a surface consisting of several hundred polygons.

Figure 5.5
Three-dimensional shrub modeled using L-systems
(Courtesy of P. Prusinkiewicz)

5.5 Stochastic and Parameterized Systems

The L-systems introduced so far are deterministic, and thus always produce the same branching structures. In order to replicate the variability of nature, probabilistic influences must be brought into the produced structures. This can be achieved, for example, by means of variation of parameters such as the

deviation angle δ. In this way, however, only geometrical characteristics of the models are modified. In order to modify the topological characteristics, the application of rules must occur at random.

In this procedure, P defines several rules with the same left side ϕ. Additionally, as already shown at the beginning, per rule an application probability is given. Using a random generator, the text of the rule is applied, according to its probability, during expansion.

Additionally, in a parametric L-system all parameters can be modified dynamically during expansion. The commands are defined for this step in parameterized form, whereby per instruction one or more parameters are used. The parameters are summarized in the parameter vector **w**, their meaning depends on the application.

On the right side of each rule, the parameters in the numeric expressions can be used for the computation of new parameter values. The instructions in parameterized form, also called modules, have the following forms: ← module

F(**w**) Move turtle at $\vec{d}(\mathbf{w})$ in current direction, draw line:
$$(x, y, z, \mathbf{M}) \rightarrow (x + (\mathbf{M}\vec{d}(\mathbf{w}))_1, y + (\mathbf{M}\vec{d}(\mathbf{w}))_2, z + (\mathbf{M}\vec{d}(\mathbf{w}))_3, \mathbf{M})$$

f(**w**) Move turtle at $\vec{d}(\mathbf{w})$ in current direction, without drawing line:
$$(x, y, z, \mathbf{M}) \rightarrow (x + (\mathbf{M}\vec{d}(\mathbf{w}))_1, y + (\mathbf{M}\vec{d}(\mathbf{w}))_2, z + (\mathbf{M}\vec{d}(\mathbf{w}))_3, \mathbf{M})$$

&(**w**) Increase angle in terms of lateral axis at $\mathbf{w}$:
$$(x, y, z, \mathbf{M}) \rightarrow (x, y, z, \mathbf{M} \cdot \mathbf{R_x}(\mathbf{w}))$$

+(**w**) Increase angle in terms of the vertical axis at $\mathbf{w}$:
$$(x, y, z, \mathbf{M}) \rightarrow (x, y, z, \mathbf{M} \cdot \mathbf{R_y}(\mathbf{w}))$$

\(**w**) Increase angle in terms of roll axis (longitude) at $\mathbf{w}$:
$$(x, y, z, \mathbf{M}) \rightarrow (x, y, z, \mathbf{M} \cdot \mathbf{R_z}(\mathbf{w}))$$

The reduction of the angles is made possible by negative values for **w**, which make some commands redundant. A simple example produces an abstract, binary tree:

Let $n = 10$, $\delta = 85°$, and $R = 1.456$, a constant selected to be slightly above $\sqrt{2}$, and use the axiom $w = A(1)$. Using the production

$$A(s) ::= F(s)[+A(s/R)][-A(s/R)]$$

a tree can be constructed successively. In Fig. 5.6 the result is presented, where the angle δ ensures that the tree is not developed in a right-angled way and the constant R limits the size of the subtrees in such a way that the branches just do not overlap.

Parametric L-systems are also Prusinkiewicz and Lindenmayer's answer in response to the assertion of Honda as well as that of Aono and Kunii (see also Sect. 4.3). Both stated at the beginning of the 1980s that complex trees can not

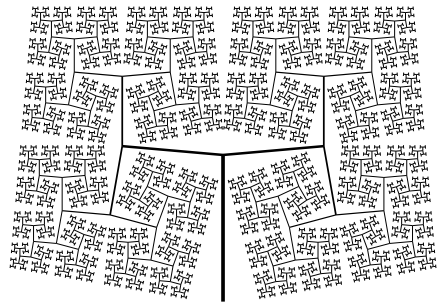


Figure 5.6
Binary tree, generated using a parameterized L-system

be produced using L-systems. In Fig. 5.7a–b two trees can be seen that replicate the branching mechanism of Honda using parametric L-systems; in Fig. 5.7c the method of Aono and Kunii is copied; in (d) an additional branching structure is shown.

(a) (b) (c) (d)

Figure 5.7
Branching structures, generated using parameterized L-systems (Courtesy of P. Prusinkiewicz)

We now turn to a further modeling option that is offered through L-systems: the production of context-sensitive structures. This method is needed particularly for the reproduction of spatial and temporal signal processes, and is a powerful aid for the construction of complex systems.

5.6 Context-Sensitive Systems

As already mentioned at the beginning of this chapter, in a context-sensitive system the use of a production is made dependent on the local environment of a character. The left side of a production now has the form $\tau A \upsilon$, where $\tau, \upsilon \in V^+$ and $a \in V$. With this type of system, it becomes problematic if several productions with differently sized contexts can be applied to one character. In order to yield a well-defined approach, additionally it is defined that the production with the largest context is used first in such a situation.

Traditionally, in connection with L-systems a context-sensitive rule is indicated in the format $\tau < A > \upsilon ::= b$. In contrast to the Chomsky notation, $\tau a \upsilon ::= b$ the context is here explicitly defined. We know what in a production is the active part that is to be replaced. A simple example is a system for the movement of a character (a signal) within a string: Let $\omega = baaaaaaa$ and

$P = \{b < a ::= b, b ::= a\}$. The result is the following sequence of strings:

$$
\begin{aligned}
&baaaaaaa\\
&abaaaaaa\\
&aabaaaaa\\
&aaabaaaa\\
&aaaabaaa\\
&\ldots
\end{aligned}
$$

Here, the character moves through the string from left to right. Such signal characters can control the growth in parts of the plant, such as the successive blooming of blossoms in a large flower stand, or they can influence the withering of subtrees in a larger tree.

(a) (b) (c)

Figure 5.8
Branching structures generated using context-sensitive L-systems by Hogeweg and Hesper [88] (Image courtesy of P. Prusinkiewicz)

Part	(a)	(b)	(c)
n	30	24	26
δ	22.50	25.75	22.50
Ignore:	+-F	+-F	+-F
ω	F1F1F1	F0F1F1	F1F1F1
$0 < 0 > 0 ::=$	1	1	0
$0 < 0 > 1 ::=$	1[-F1F1]	0	1[-F1F1]
$0 < 1 > 0 ::=$	1	0	1
$0 < 1 > 1 ::=$	1	1F1	1
$1 < 0 > 0 ::=$	0	1	0
$1 < 0 > 1 ::=$	1F1	1[+F1F1]	1F1
$1 < 1 > 0 ::=$	1	1	1
$1 < 1 > 1 ::=$	0	0	0
$* < - > * ::=$	-	-	-
$* < + > * ::=$	+	+	+

If branching structures are treated, then the mechanism must ignore the parts included in the parentheses for the determination of the context, because otherwise the structural context of the characters cannot be found. Also, special

characters should be ignored for controlling the geometrical interpretation; this is indicated together with the definition of the system.

In Fig. 5.8 several context-sensitive branching structures are shown. The results originate from a work by Hogeweg and Hesper [88] in which systematically a number of relatively simple context-sensitive systems were analyzed. Smith [201] later refined the graphical representation. The illustrations are taken from [166].

When comparing the systems in Fig. 5.8, it is noticeable that the systems (a) and (c) are very similar, although the produced structures have very different appearances. This also explains why the results were obtained only using a systematic search. Nevertheless, it is unquestionable that only with context-sensitive systems processes such as signal distributions can be modeled in plants. However, at least for the definition of branching structures there seem to be alternatives. With exception rules and indexed items that assign an unique identification number at any time to each of the produced partial objects, similar results can be obtained (see Sect. 6.5).

exceptions →

indexing →

However, such an indexing has the disadvantage that in an animation the number of objects in a plant can change and that then also the indices are again newly assigned, at least if the indexing is done automatically. This again would cause the exceptions (inasmuch as they are marked by indices) to affect other parts of the objects, which must be avoided at all cost. To the reader perhaps this appears as an unimportant detail problem; nevertheless, there are a number of consequences for practical work. Therefore, index-free L-systems have here a primary advantage.

5.7 Modeling of Phyllotaxis

Phyllotaxis is the spatial arrangement of leaves in blooms and was introduced in Sect. 3.4. Also, we previously discussed different procedures for the production of phyllotaxial patterns. In the following, a solution with L-systems is illustrated.

Golden Section →

We will therefore return to the arrangement of the seeds in a sunflower, which was likewise addressed in Sect. 3.4. With a parametric L-system the typical angle of rotation is modeled according to the Golden Angle of $137.5°$ in combination with the square root increase of the radius (see also [66]). Thereby D is the L-system for the description of a small circle, and $w = A(n)$:

$$A(n) ::= +(137.5)[f(\sqrt{n}) \sim D]A(n+1).$$

Accordingly, a complete head of a sunflower can be generated, as shown in Fig. 5.9, with the aid of L-systems for the creation of the form of the seeds S, the inflorescence R, and the petals M, N, O, and P:

$$\begin{aligned} &A(n) ::= +(137.5)[f(\sqrt{n})C]A(n+1) \\ &C(n) : n \leq 440 \qquad ::= \sim S \end{aligned}$$

$$
\begin{aligned}
C(n) &: 440 < n \leq 565 ::= \sim R \\
C(n) &: 565 < n \leq 580 ::= \sim M \\
C(n) &: 580 < n \leq 595 ::= \sim N \\
C(n) &: 595 < n \leq 610 ::= \sim O \\
C(n) &: 610 < n \qquad\quad ::= \sim P
\end{aligned}
$$

The system differentiates the six forms to be produced according to "the control variable" n (in this case also an index). If the control variable is smaller than 440, seeds are produced, then inflorescences, and finally different forms of petals.

Figure 5.9
Realistic plant models, generated using parameterized L-Systems (Courtesy of D. Fowler, P. Prusinkiewicz)

Contrary to the sample models in Sect. 4.13, the L-system again is a descriptive model, which does not relate to natural processes, but only approximates the outer appearance of the corresponding object.

5.8 Animation of L-Systems

In [166] and later in [164] Prusinkiewicz et al. introduced extensions for L-systems which allow for the animation of growth processes in static models. The most important extensions are the so-called timed L-systems, in which a number is assigned to each character and to each module. In the left side of the production the number represent the terminal age of a module, in the right side they represent the initial age. Within the animation, the global time will be counted up, and for the terminal age of the modules the appropriate productions are applied. The life spans of the modules start with their local initial ages and, if a terminal age is reached, the rewriting of that module occurs and new modules are produced.

While timed L-systems permit the animation of growth procedures, they do not permit communication of co-existing modules, which is nevertheless an important condition for the development of natural branching structures, as was already shown in Sect. 4.3. In [164] differential L-systems (DL-systems) are introduced that make this communication possible. Here L-system-like productions describe the topological modifications of the models, and differential equations describe the changes during the advance of time. According to their

Figure 5.10
Growth of Hieracium umbellatum (Courtesy of M. Hammel, P. Prusinkiewicz)

nature, DL-systems are context-sensitive 2L-systems, i.e., the left side of the productions has the form $\phi = \tau a \upsilon$, where $\tau, \upsilon \in V^+, a \in V$. Since these are parametric 2L-systems, a parameter set is assigned to each module **w**:

$$\phi = \tau(\mathbf{w}_\tau)a(\mathbf{w})\upsilon(\mathbf{w}_\upsilon). \tag{5.1}$$

differential L-systems →

The continuous behavior $a(\mathbf{w})$ is described by a differential equation that describes the rate of change $d\mathbf{w}/dt$ of the parameter set **w** as a function of the current values of **w** as well as those of their neighbors $(\mathbf{w}_\tau, \mathbf{w}_\upsilon)$:

$$\frac{d\mathbf{w}}{dt} = f_a(\mathbf{w}_\tau, \mathbf{w}, \mathbf{w}_\upsilon). \tag{5.2}$$

Equation (5.2) has a definition range D_a, which is determined by the module a. D_a is limited by a boundary C_a, that consists of m non-intersecting curves $C_{a_k}, k = 1, 2, ..., m$. For each of these boundaries a production exists in P with

$$\tau(\mathbf{w}_\tau)a(\mathbf{w})\upsilon(\mathbf{w}_\upsilon) ::= \chi_k(\mathbf{w}_k), \tag{5.3}$$

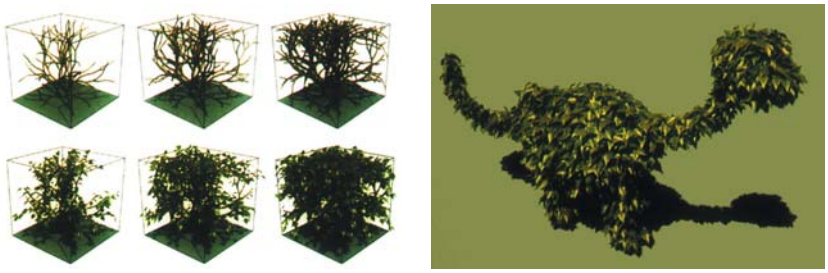
which is applied if the path (the change of values) of the **w** over time reaches the boundary C_{a_k}. In this case, a function h_{a_k} is used, which determines the new values $\mathbf{w}_k$:

$$\mathbf{w}_k = \lim_{t \to t_c} h_{a_k}(\mathbf{w}_\tau(t).\mathbf{w}(t), \mathbf{w}_\upsilon(t)) \tag{5.4}$$

The new parameter vector $\mathbf{w}_k$ must belong to the domain D_{χ_k} to continue the process of this domain. Altogether each module a is described by the vector $\{D_a, C_a, f_a, P_a\}$, which lets us suspect that we are dealing with a very complex system. Figure 5.10 shows, however, that the mechanism allows for the animation of complex botanical processes.

5.9 Interactions of Plants and the Environment

The parametric L-systems introduced so far are able, due to their context-sensitivity, to produce situation-dependent topologies and geometry; however, a real interaction of plants and their environment is at this time not really



(a) (b)

Figure 5.11
Pruning of plants to a box shape and to an implicit surface (Courtesy of R. Mech, M. James, P. Prusinkiewicz)

possible. Two works were presented addressing this topic: [165] introduced environment-sensitive L-systems, and [144] proposed open L-systems.

In an environment-sensitive L-system, local aspects of the environment can affect the model. To model the effects, inquiry symbols of the form $?X(x, y, z)$ with $X = P, \vec{H}, \vec{U}, \vec{L}$ are incorporated into the language. At the time of interpretation of the string, these symbols determine the position or orientation. In the first case ($X = P$) the values x, y, z represent the location, in the second case ($X = \vec{H}, \vec{U}, \vec{L}$) the x, y, z represent the coordinates of the direction vector in the coordinate system. These values are passed on to user-defined functions, where their resulting values in turn affect the continuation of the string interpretation.

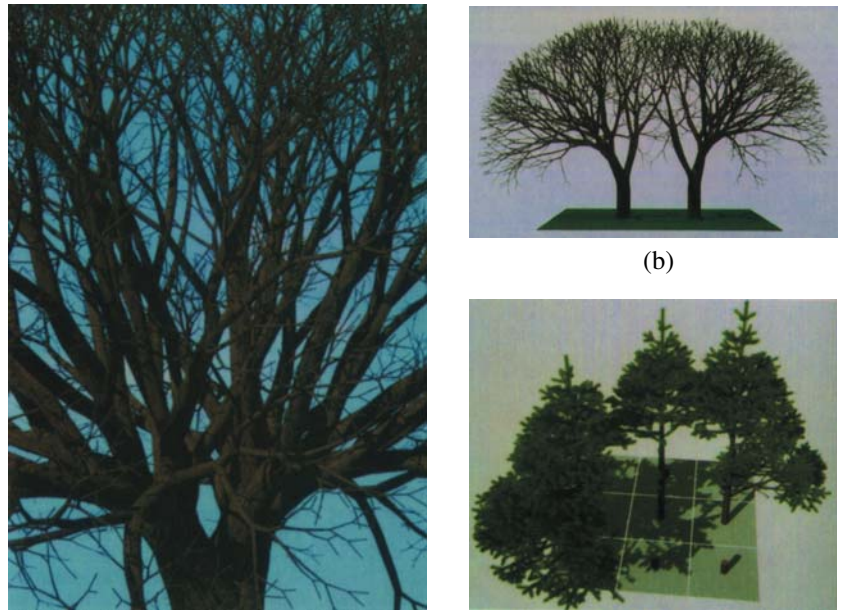
Using this mechanism, pruning can be implemented (see Fig. 5.11a). Another option is to limit the growth of the plant to the surface of other objects. The object is here defined as an implicit surface (see Fig. 5.11b).

In an open L-system, communication is no longer one-sided, but rather bidirectional. Those parameters connected with a communication symbol can be set either by the environment and passed on to the model or they can be set by the model and are passed on to the environment.

Communication was limited in the environment-sensitive L-systems to an inquiry on position and orientation. Using open L-systems, communication modules of the form $?E(x_1, ..., x_m)$ for sending and receiving parameters are applied. Here, the produced text is just like in normal L-systems, evaluated from left to right so as to designate the state of the turtle for each symbol. In this process, however, no character operations are implemented. If a communication symbol is reached, the geometry-generating process sends a message to the environment modeling process with the following parameters:

- address of the communication module in the string
- values of $x_1, ..., x_m$
- state of the turtle (position, orientation)
- type and kind of module on the right side of the communication module

The messages are read by the environment modeling process and worked on after conclusion of the interpretation of the string. The new values for the vari-



(a) (b) (c)

Figure 5.12
Interaction of plants with the environment: (a) branching structure with interaction; (b) two merging trees; (c) interacting group of trees (Courtesy of R. Měch, P. Prusinkiewicz)

ables $x_1, ..., x_m$ are then sent back to the communication modules. Here the addresses of the communication modules are used. In this way, theoretically the entire information in the plant can be passed on to the environment; usually, however, only a part of the information is needed.

In Fig. 5.12a this mechanism is used for modeling a branching structure. The smaller branches interact with one another, in order to not exceed a defined density. This follows the branching model of Honda from Sect. 4.3 and is influenced by the light. If two trees are planted next to each other, their crowns form one system (Fig. 5.12b). A tree group can likewise be modeled as an interacting system (Fig. 5.12c). The appropriate systems are relatively large, and are described as in [144].

5.10 Use of Position Information

Similarly to how environmental information is incorporated into the production of the branching structures, geometrical aspects of modeling can be integrated as well [167]. Instead of a communication module, user-defined functions are inserted. To obtain natural effects, a small number of B-spline curves is sufficient.

Figure 5.13 shows the successive construction of a compound leaf. The user defines the leaves by first entering their length, then their size, the branching angle, the style form, and, finally, by adding a random offset angle (twisting).

With this method, the disadvantage of the predominantly local modeling methodology can be partially eliminated using L-systems, since the position infor-

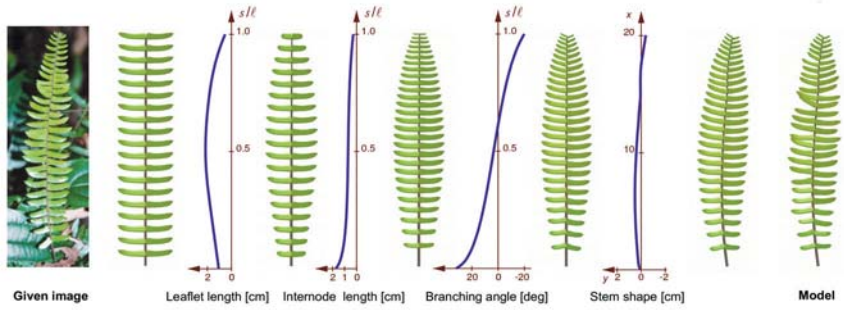


Figure 5.13
Editing using B-spline curves (Courtesy of P. Prusinkiewicz)

mation of the objects permits a function inquiry, and thus allows the user to integrate global aspects into the production. However, the formulation of the productions seems quite laborious.

In addition to modeling with Lindenmayer systems, Prusinkiewicz et al., extend the formalism using sequentially processed Chomsky grammars, which are applied in the simulation of structurally independent plant parts. Hereby, the L-systems regulate the exchange of materials and time-dependent processes.

In conjunction with the introduction of an interactive editor for adjustment of the functions, Prusinkiewicz et al. get closer to the efficient modeling of plants. This applies also for the Xfrog system described in the next chapter. However, beforehand we will introduce a few additional approaches that, although they work with rule-based systems, use a different modeling methodology.

5.11 Iterated Function Systems

The method proposed by Barnsley and Demko [10, 35] for modeling natural objects does not work with geometric data, but rather with two- or higher-dimensional point sets. The structures modeled with this method therefore represent only images of the objects, and no surfaces, as was the case in the preceding approaches.

To define an Iterated Function System (IFS) n contractive, affine transformations $M_j \in \mathcal{R}^2 \rightarrow \mathcal{R}^2$, $M = \{M_1, M_2, ..., M_n\}$ are required. Each of the affine transformations has six parameters a–f, and is defined by:

$$M_j(x, y) = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} e \\ f \end{pmatrix}. \tag{5.5}$$

Here point (x, y) is moved to another position. The sequential application of the transformations generates a trace of the point. While the trace is not particularly interesting when applying the transformation only once, amazingly enough complex patterns evolve during the sequential application of several transformations M_j.

The attractor of the IFS is the smallest nonterminal subset $A \in I\!R^2$, so that $M_j(\mathbf{p}) \in A$ for all $p \in A$ and $1 \leq j \leq n$. The set A always exists and

is unambiguous (see [168]). In order to approximate A, one takes any point $\mathbf{p}_s \in A$ and combines the point sets that emerge if the transformations M_j are applied to the point. If $\mathbf{p}_s$ is not in A, then due to the contraction characteristics of M_j it is traced back to the attractor.

A can be generated in different ways. Deterministic procedures usually develop a tree, whose nodes consist of points A, and whose edges are applications of M_j. If for each point each function M_j is applied, a binary tree develops for $j = 2$; for larger values an accordingly broader tree is generated. The tree represents the set of all points producible from one point, and grows exponentially with the number of function applications. Should several initial points exist, then, of course, there will be a tree for each of them.

In order to produce an interesting image as fast as possible out of this vast set of potential function calls, different search strategies are in place. One of them is the depth-first search, in which, outgoing from the initial point, a function is applied, and following the result, is immediately succeeded by the next. Hence, it is possible to move upward in the tree very quickly. Alternatively, using breadth-first search, for a certain point all applicable transformations are applied. In this way, step by step the complete tree is constructed.

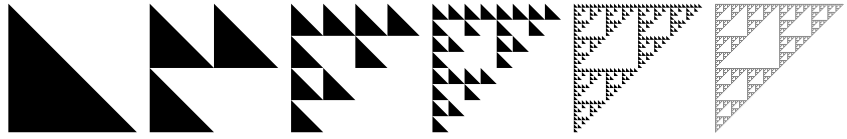


Figure 5.14
Production of a Sierpinski triangle using deterministic breadth-first search (depth 0, 1, 2, 3, 6, 9)

An example of this algorithm is the construction of a Sierpinski triangle. Here for the production of A the breadth-first search is applied, whereby simultaneously an entire point set is processed. Three functions are needed, which in each case have a contraction factor of 0.5 with differing individual displacements of the input points:

j	a	b	c	d	e	f
1	0.5	0	0	0.5	0	0
2	0.5	0	0	0.5	0	50
3	0.5	0	0	0.5	50	50

Figure 5.14 illustrates the result of the application of the functions to a triangular point set. The sides of the triangle are divided by two in each case, and the reduced triangle is moved in three different places.

To work with balanced trees, defined completely on each level, is a disadvantage if the illustrations M_j contain different contraction rates s_j (the so-called Lipschitz constants). These are defined by

Lipschitz constant →

$$\| M_j(\mathbf{p}_1) - M_j(\mathbf{p}_2) \| \leq s_j \| \mathbf{p}_1 - \mathbf{p}_2 \| \qquad \forall \mathbf{p}_1, \mathbf{p}_2 \in I\!R^2. \tag{5.6}$$

For such unequal Lipschitz constants, balanced scanning along the tree leads to an unequal point distribution in A. With an appropriate truncation of subtrees

this can be avoided. Figure 5.15a illustrates the effect of the uneven distribution and the associated visually small convergence. A tree depth of nine and 262,144 points were used in order to assess the following IFS:

j	a	b	c	d	e	f
1	0	0	0	0.16	0	0
2	0.85	0.04	-0.04	0.85	0	1.60
3	0.20	-0.26	0.23	0.22	0	1.60
4	-0.15	0.28	0.26	0.24	0	0.44

The function M_1 has a strong contraction rate, M_3 and M_4 are medium strong contractive, while M_2 hardly contracts. If subtrees are appropriately truncated, then an even accumulation of points results, as shown in Fig. 5.15b for the same number of points.

Figure 5.15
Generating a fern leaf with 262,144 points each: (a) deterministic method using a balanced tree; (b) deterministic method with unbalanced tree; (c) stochastic method

(a) (b) (c)

Another method introduced by Barnsley [9] consists of a stochastic depth-first search of the tree, whereby a function is selected randomly each time and is then applied. In order to improve the convergence, a similar procedure to that used with nonbalanced trees is applied, meaning that per function M_j a probability $P_j, \sum_{j=1}^{n} P_j = 1$, is defined which indicates how often M_j should be applied. If P_j is set according to the Lipschitz constant of the corresponding function, we obtain convergence characteristics as good as those obtained with the truncation method. For the above example, we apply $P_1 = 0.01, P_2 = 0.85, P_3 = 0.07, P_4 = 0.07$. In Fig. 5.15c the result is shown with again 262,144 points.

In his book "Fractals Everywhere" [9] Barnsley extends the method and applies it to simulate images of natural scenes. In such an image many attractors are combined, and also here hierarchical IFSs are used that are not distributing points, but rather whole subimages. The method has the disadvantage that finding appropriate subimages can be a very cumbersome task. Often, only manual intervention yields satisfactory results. If such functions are found, then the attainable compression factors are, however, very high. As an extension of the procedure, if Iterated Function Systems are applied, it is even possible to convert arbitrary input pictures into arbitrary final results.

As an additional extension of this approach, RIFSs (recurrent IFSs) are introduced, in which no arbitrary transformation may follow another, but rather pairs of sequential transformations are specified. This usually is implemented using a graph, whose nodes indicate the transformations, and whose edges denote the possible successor. The attractors of the RIFSs are therefore subsets of the attractors of common IFSs.

Prusinkiewicz and Lindenmayer in [166] describe a procedure for the transformation of L-systems into RIFSs. This is successful if the L-systems describe plants with a strict self-similarity. The reverse, to generate from RIFSs a parametric L-system, is rather simple. Nevertheless, the affine transformations in the parameters must be coded accordingly.

5.12 Object Instancing

To model branching structures, often tree-like data structures are used. We saw this already in some procedural methods in Chap. 4. In the case of L-systems, the tree is represented as text in linear form. Iterated function systems can, as demonstrated above, likewise be represented as trees that encode the transformation rules for points.

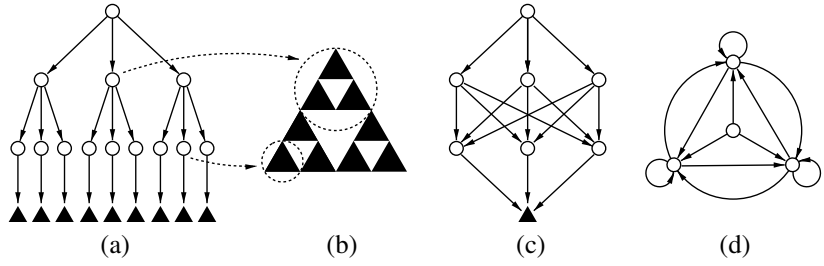


Figure 5.16
Object instancing of fractal objects: (a) tree structure; (b) resulting approximation of a Sierpinski-triangle; (c) object instancing hierarchy; (d) object instancing hierarchy with cycles

Since these data structures most likely occupy a lot of space, depending upon the complexity of the objects, it is meaningful to introduce compression techniques. One option is to represent the produced geometrical primitives canonically and to place them in a tree using relative positioning data. In this case, subtrees, which produce the same objects in different places, can be represented by just one subtree, whose root nodes may have several fathers. The tree becomes in this case a directed acyclic graph (DAG). The process is called object

instancing, and was used for the first time by Ivan Sutherland in his system "Sketchpad" [216].

A subset of L-systems, as well as the iterated function systems, can efficiently be represented by such graphs [166]. Kay and Kajiya [108] applied this technology for the first time to fractal objects. Here, cycles develop in the directed graphs, and the object can be described in arbitrary detail.

Hart and DeFanti [84] describe efficient methods for the modeling and representation of cyclic object instancing hierarchies. Figure 5.16a shows the tree hierarchy for modeling an approximation of the Sierpinski triangle from part (b). The leaves of the tree are the geometric primitives; the internal nodes represent relative transformations and combine the produced geometry.

In Fig. 5.16c the graph was converted into a directed acyclic graph that also represents the object. If cycles in the graph are permitted (Fig. 5.16d), then the fractal object can be represented arbitrarily exactly and a Sierpinski triangle is yielded. Hart [83] provides a more detailed description of the graph in Fig. 5.16d:

```
canonical object up                    // upper node
{       triangle                       // draw this, if recursion
        merge: up, left, right         // stops
        scale: 0.5, 0.5
        move: 0, 0.5
}

canonical object left                  // left node
{       triangle
        merge: up, left, right
        scale: 0.5, 0.5
        move: –0.433, –0.25
}

canonical object right                 // right node
{       triangle
        merge: up, left, right
        scale: 0.5, 0.5
        move: 0.433, –0.25
}

object top                             // node in the middle
{       merge: up, left, right
}
```

Likewise in [83] the transformation of a subset of L-systems and of RIFSs into the cyclic graphs of the object instancing paradigm is described. The conversion of the RIFS is executed via the transformation of each function into a

canonical object; the RIFS graph is transferred directly into the cyclic graph of the object hierarchy.

The already-mentioned possibility of converting L-systems into RIFSs (see [166]) allows us also to convert them into the object instancing paradigm. Hart sketches furthermore a direct conversion method for strongly recursive branching structures. However, a general transformation method seems not to exist.

Instancing is possible within a branching structure as well as in a compilation of plants. Thus in Chap. 8 instancing is applied for the modeling of plant communities. Instead of defining each plant individually in a lawn, a small number of prototypes is stored and copied as needed. This permits a strong reduction of the needed disk space, and at the same time makes the scenes easier to handle.

A substantial difference to the method described here is the approximate character of instancing. A number of similar objects are represented visually by only one prototype. "Approximate instancing" can be used for natural objects on many levels, without substantially affecting the visual appearance of the results.

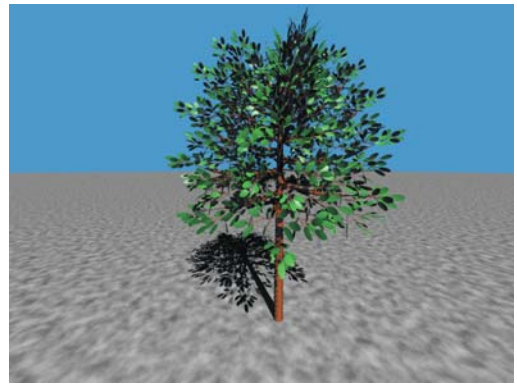


Figure 5.17
Tree model generated using a cyclic CSG-graph (Courtesy of M. Gervautz and C. Traxler)

5.13 CSG-Based Modeling

Object instancing is also used in CSG graphs (Constructive Solid Geometry), a common type of representation for objects within the CAD domain. A CSG expression is a set operation with volume objects or with point sets in $\mathbb{R}^3$. The basic objects (ball, cylinder, cone, etc.,...) are canonically represented, transformed individually, and combined using Boolean operations ($\cap, \cup, \setminus$, etc.,...).

Since the Boolean operations are univalent or bivalent (unary or binary), the expression can also be illustrated in the form of a binary tree (CSG tree). Gervautz and Traxler [74] extend this tree description by allowing cyclic CSG graphs, and by permitting arbitrarily refined descriptions of objects. A CSG graph can be regarded as an extension of object instancing, since aside from the union operation other Boolean operations are allowed.

In the approach of Gervautz and Traxler, the CSG graphs are produced by parametric L-systems. Due to the special structure of the graphs, the languages have the following notation (axiom ω: E):

$$
\begin{aligned}
E &::= E \text{ op } E && \text{// op} \in \{\cap, \cup, \backslash\} \\
E &::= \text{trans } E && \text{// trans} \in \{\text{rotx}(\alpha), \text{roty}(\alpha), \text{rotz}(\alpha), \\
& && \text{// } \quad \text{move}(dx, dy, dz), \text{scale}(dx, dy, dz) \\
& && \text{// } \quad \text{uscale}(s)\} \\
E &::= (E) \\
E &::= \text{obj} && \text{// obj} \in \text{set of primitive objects}
\end{aligned}
$$

The parameterized description of the Sierpinski triangle or its discrete approximation with a CSG tree of depth six is represented in this notation as (with $\omega = S(6)$):

$$
\begin{aligned}
S(c) : c = 0 &::= \text{triangle} \\
S(c) : c > 0 &::= \text{move}(0.5, 0.5)\text{uscale}(0.5)S(c-1) \\
&\quad \cup \text{move}(0.5, -0.5)\text{uscale}(0.5)S(c-1) \\
&\quad \cup \text{move}(-0.5, 0.5)\text{uscale}(0.5)S(c-1)
\end{aligned}
$$

For the transformation of the L-system into a cyclic CSG graph, additionally three node types are introduced: A *t-node* is a parameterized transformation node; an *s-node* represents an option; a *c-node* is able to implement a numeric computation. The graph for the above system is represented in Fig. 5.18a.

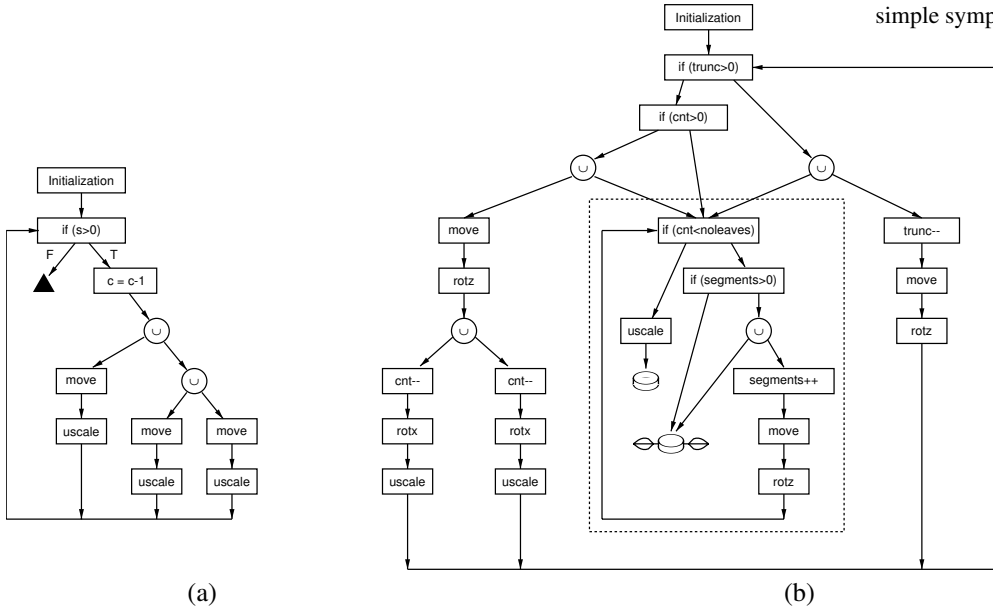


Figure 5.18
Cyclic CSG graphs: (a) description of a Sierpinski triangle; (b) description of a simple sympodial branching structure

Figure 5.18b shows yet another graph, this time for the simulation of a sympodial branching structure. The appropriate L-system with its parameters can

be found in [74]. The trees which can thereby be produced correspond in their optical quality to those generated using simple procedural methods.

5.14 Categorizing the Methods

Rule-based systems and in particular L-systems provide possibilities for modeling plants in extremely compact ways. The fundamental formalism of L-systems is simple and has its aesthetic attraction. Using various extensions, such as context-sensitive, parameterized, differential as well as open L-systems, Prusinkiewicz et al. succeeded in adding to formalism a wide range of options that allow for numerous branching patterns to be modeled. In [197] it was shown that with L-systems principally all 23 tree architectures of Hallé, Oldeman and Tomlinson [81] can be simulated. Together with the additional functions that can be entered by the user, and hence the option of interactive modeling, the modeling process became a much easier task.

Also the visual results converged over the course of the time with the quality of procedural methods. Today models of similar quality can be produced with either approach. It should also be emphasized that the extraordinary spectrum of possible plant models is not restricted to trees. During the development of the discussed methodology, all types of plants were modeled. This differentiates the approach clearly from the procedural methods that were designed especially for particular structures.

However, the modeling aspect is still problematic. As already previously noted in this chapter, the production of a given plant with an L-system is a difficult process. The user must be proficient in order to represent a plant from the very start using an L-system. Many aspects of the plant have to be defined locally, and already small changes sometimes cause a complete modification of the total shape. In addition, the production of plant geometry is a computationally expensive task. After each parameter change, a complete expansion process must be worked through, including the geometry production, which significantly slows down progress. This is especially disturbing when working with huge models such as trees. However, with the improvement of computer efficiency, this becomes a less significant factor.

Likewise, the graph-based procedures discussed above do not contain any improvements with regard to the modeling options of rule-based procedures. With these methods, only part of what is definable with the L-systems is actually converted. The possibility of representing the topology of plants over graph structures is, however, an interesting aspect. The “rules” in this case denote instancing dependencies in the graph: if a node is arrived at, and if appropriate geometry is created, then transformations of the local coordinate system are invoked, and the geometries of all child components connected with the node are produced.

In the next chapter this mechanism is extended in order to store in a graph structure a part of the plant topology. The nodes of the graph are now com-

ponents that contain data and algorithms in order to procedurally create parts of the plants. The combination of a graph-based rule system and procedural methods combines the advantages of both methods.